The *functional acceptance tests* are business-facing tests that support development teams, ensuring they are building the *right software*. They take on the user perspective and are usually implemented via *executable specifications* using a high-level domain-specific language (DSL), which is then translated into a lower-level programming language. For example, you could use Cucumber (https://cucumber.io) to write scenarios like "browse the book catalog" or "place a book order" in human-friendly plain text. Those scenarios can then be executed and verified using a programming language like Java.

In the acceptance stage, we can also verify the *quality attributes* of a release candidate via *non-functional acceptance tests*. For example, we could run performance and load tests using a tool like Gatling (https://gatling.io), security and compliance tests, and resilience tests. In this last case, we could embrace *chaos engineering*, a discipline made popular by Netflix and consisting of making certain parts of the system fail to verify how the rest will react and how resilient the system is to failures. For Java applications, you can look at Chaos Monkey for Spring Boot (https://codecentric.github.io/chaos-monkey-spring-boot).

NOTE How about the third quadrant? Following the continuous delivery principles, we strive not to include manual tests in the deployment pipeline. Yet we usually need them. They are particularly important for software products aimed at end users like web and mobile applications. Therefore, we run them on the side in the form of *exploratory testing* and *usability testing*, so that we ensure more freedom for testers and fewer constraints on the pace and timing required by continuous integration and the deployment pipeline.

An essential feature of the acceptance stage is that all tests are run against a production-like environment to ensure the best reliability. The deployment would follow the same procedure and scripts as production and could be tested via dedicated system tests (first quadrant).